

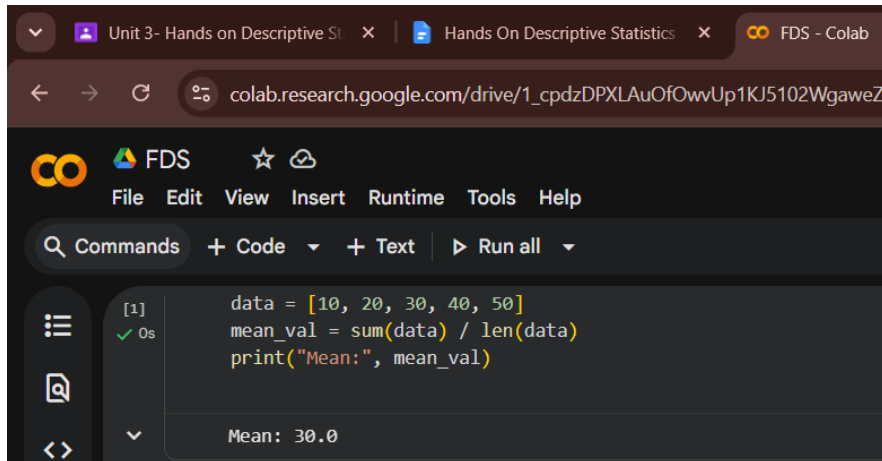
1. Finding Mean Using Python

python

```
data = [10, 20, 30, 40, 50]
```

```
mean_val = sum(data) / len(data)
```

```
print("Mean:", mean_val)
```



2. Finding Median Using Python

python

```
data = [40, 10, 30, 50, 20]
```

```
data.sort()
```

```
n = len(data)
```

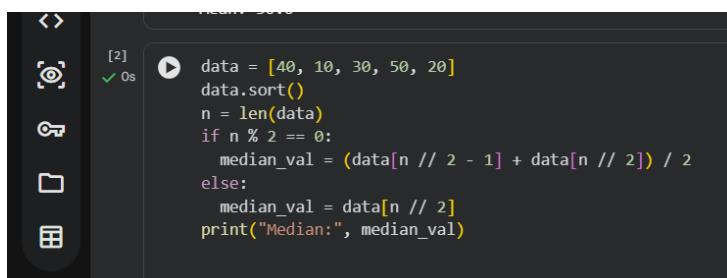
```
if n % 2 == 0:
```

```
    median_val = (data[n // 2 - 1] + data[n // 2]) / 2
```

```
else:
```

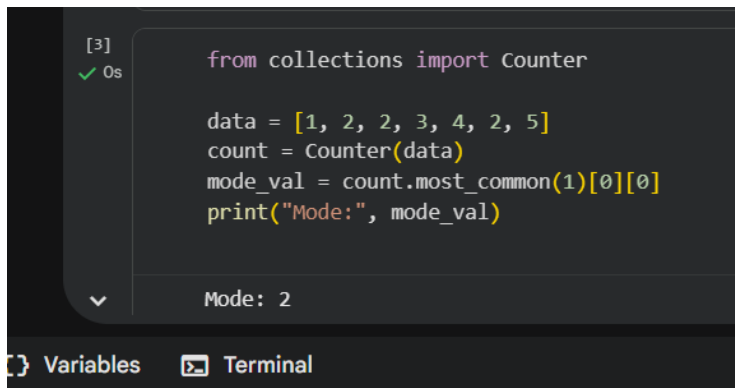
```
    median_val = data[n // 2]
```

```
print("Median:", median_val)
```



3. Finding Mode Using Python

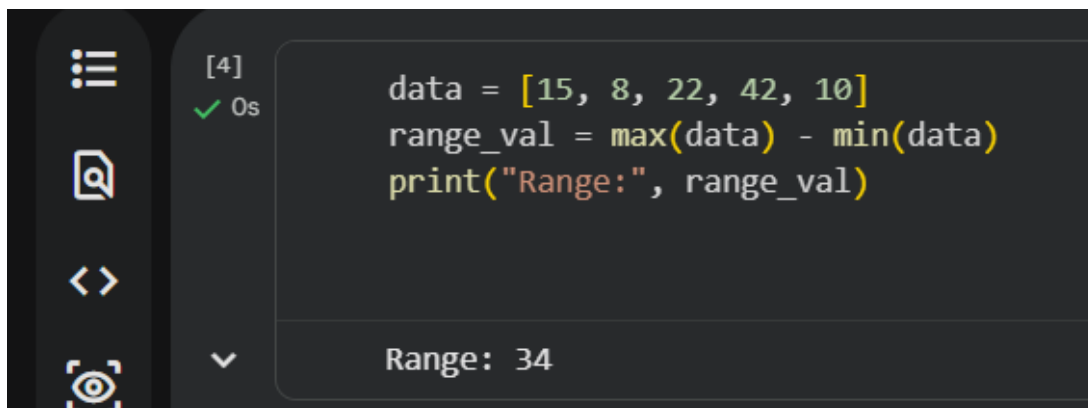
```
from collections import Counter  
  
data = [1, 2, 2, 3, 4, 2, 5]  
  
count = Counter(data)  
  
mode_val = count.most_common(1)[0][0]  
  
print("Mode:", mode_val)
```



The screenshot shows a Jupyter Notebook interface. On the left, there is a sidebar with icons for a menu, search, code, and a console. The main area displays a code cell with the following Python code: `from collections import Counter`, `data = [1, 2, 2, 3, 4, 2, 5]`, `count = Counter(data)`, `mode_val = count.most_common(1)[0][0]`, and `print("Mode:", mode_val)`. Below the code, the output is displayed as `Mode: 2`. The cell is labeled with `[3]` and `✓ 0s` in the top left corner.

4. Finding Range Using Python

```
data = [15, 8, 22, 42, 10]  
  
range_val = max(data) - min(data)  
  
print("Range:", range_val)
```



The screenshot shows a Jupyter Notebook interface. On the left, there is a sidebar with icons for a menu, search, code, and a console. The main area displays a code cell with the following Python code: `data = [15, 8, 22, 42, 10]`, `range_val = max(data) - min(data)`, and `print("Range:", range_val)`. Below the code, the output is displayed as `Range: 34`. The cell is labeled with `[4]` and `✓ 0s` in the top left corner.

5. Finding Variance Using Python (Population)

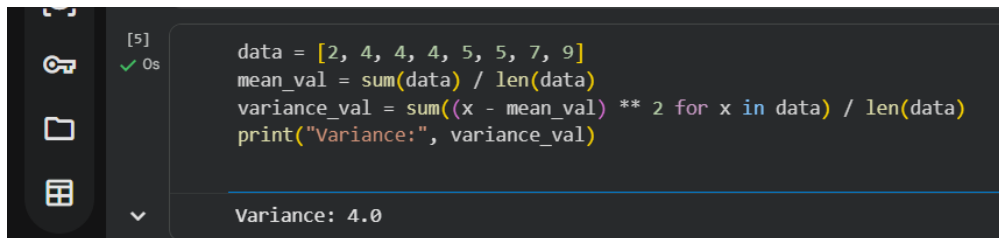
python

```
data = [2, 4, 4, 4, 5, 5, 7, 9]
```

```
mean_val = sum(data) / len(data)
```

```
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
```

```
print("Variance:", variance_val)
```



```
[5] ✓ 0s  
data = [2, 4, 4, 4, 5, 5, 7, 9]  
mean_val = sum(data) / len(data)  
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)  
print("Variance:", variance_val)  
  
Variance: 4.0
```

6. Finding Standard Deviation Pure Python

python

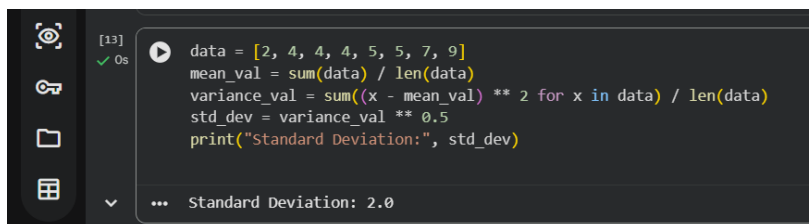
```
data = [2, 4, 4, 4, 5, 5, 7, 9]
```

```
mean_val = sum(data) / len(data)
```

```
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
```

```
std_dev = variance_val ** 0.5
```

```
print("Standard Deviation:", std_dev)
```



```
[13] ✓ 0s  
data = [2, 4, 4, 4, 5, 5, 7, 9]  
mean_val = sum(data) / len(data)  
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)  
std_dev = variance_val ** 0.5  
print("Standard Deviation:", std_dev)  
  
... Standard Deviation: 2.0
```

7. Using Built-in Module

python

import statistics

```
data = [12, 15, 15, 18, 22, 25]
```

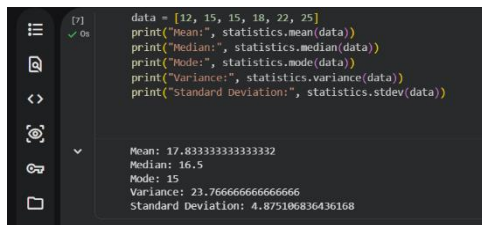
```
print("Mean:", statistics.mean(data))
```

```
print("Median:", statistics.median(data))
```

```
print("Mode:", statistics.mode(data))
```

```
print("Variance:", statistics.variance(data))
```

```
print("Standard Deviation:", statistics.stdev(data))
```

A screenshot of a Python IDE with a dark theme. The editor shows the following code:

```
[7] data = [12, 15, 15, 18, 22, 25]
print("Mean:", statistics.mean(data))
print("Median:", statistics.median(data))
print("Mode:", statistics.mode(data))
print("Variance:", statistics.variance(data))
print("Standard Deviation:", statistics.stdev(data))
```

The output console on the right displays the results:

```
Mean: 17.833333333333332
Median: 16.5
Mode: 15
Variance: 23.766666666666666
Standard Deviation: 4.875186836436168
```

8. Using NumPy Library

import numpy as np

```
data = np.array([10, 20, 20, 30, 40])
```

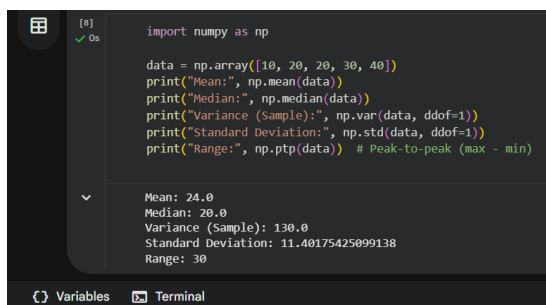
```
print("Mean:", np.mean(data))
```

```
print("Median:", np.median(data))
```

```
print("Variance (Sample):", np.var(data, ddof=1))
```

```
print("Standard Deviation:", np.std(data, ddof=1))
```

```
print("Range:", np.ptp(data)) # Peak-to-peak (max - min)
```

A screenshot of a Python IDE with a dark theme. The editor shows the following code:

```
[8] import numpy as np

data = np.array([10, 20, 20, 30, 40])
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Variance (Sample):", np.var(data, ddof=1))
print("Standard Deviation:", np.std(data, ddof=1))
print("Range:", np.ptp(data)) # Peak-to-peak (max - min)
```

The output console on the right displays the results:

```
Mean: 24.0
Median: 20.0
Variance (Sample): 130.0
Standard Deviation: 11.40175425099138
Range: 30
```

9. Using SciPy for Mode

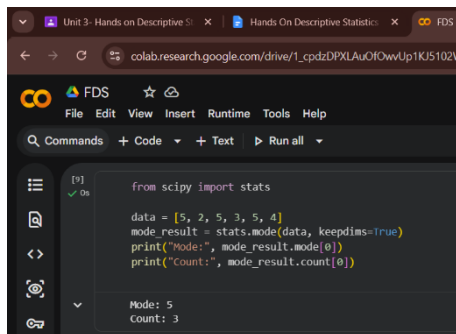
```
from scipy import stats
```

```
data = [5, 2, 5, 3, 5, 4]
```

```
mode_result = stats.mode(data, keepdims=True)
```

```
print("Mode:", mode_result.mode[0])
```

```
print("Count:", mode_result.count[0])
```

A screenshot of a Google Colab notebook interface. The top bar shows the notebook name 'Unit 3- Hands on Descriptive Statistics' and the file name 'Hands On Descriptive Statistics'. The left sidebar has icons for file explorer, code editor, and output. The main area shows a code cell with the following Python code:

```
from scipy import stats

data = [5, 2, 5, 3, 5, 4]
mode_result = stats.mode(data, keepdims=True)
print("Mode:", mode_result.mode[0])
print("Count:", mode_result.count[0])
```

The output of the code is displayed below the code cell:

```
Mode: 5
Count: 3
```

10. Using Pandas for Complete Summary

```
import pandas as pd
```

```
df = pd.DataFrame({'values': [10, 20, 30, 40, 50, 50]})
```

```
print("Mean:", df['values'].mean())
```

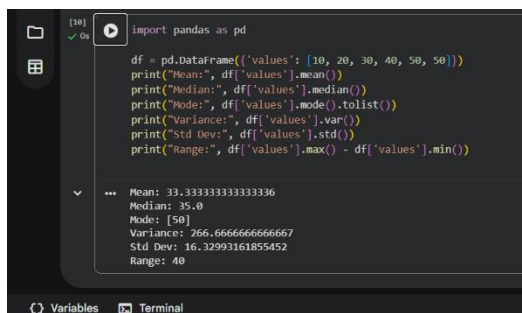
```
print("Median:", df['values'].median())
```

```
print("Mode:", df['values'].mode().tolist())
```

```
print("Variance:", df['values'].var())
```

```
print("Std Dev:", df['values'].std())
```

```
print("Range:", df['values'].max() - df['values'].min())
```

A screenshot of a Google Colab notebook interface. The top bar shows the notebook name 'Unit 3- Hands on Descriptive Statistics' and the file name 'Hands On Descriptive Statistics'. The left sidebar has icons for file explorer, code editor, and output. The main area shows a code cell with the following Python code:

```
import pandas as pd

df = pd.DataFrame({'values': [10, 20, 30, 40, 50, 50]})
print("Mean:", df['values'].mean())
print("Median:", df['values'].median())
print("Mode:", df['values'].mode().tolist())
print("Variance:", df['values'].var())
print("Std Dev:", df['values'].std())
print("Range:", df['values'].max() - df['values'].min())
```

The output of the code is displayed below the code cell:

```
Mean: 33.33333333333333
Median: 35.0
Mode: [50]
Variance: 266.6666666666667
Std Dev: 16.32993161855452
Range: 40
```